

LAB 5: OpenCL

Embedded Processing Architectures

Zhang Ruida and Hugo Bueno

May 14th, 2024

Parallelization Implementation

The basic matrix calculation algorithm is for two multiplied matrices A and B performs n multiplication and addition of n elements in row i of matrix A and n elements in column j of matrix B to obtain c_{ij} in the resulting matrix C , and repeats this process $n \times n$ times to obtain elements in the entire matrix C , where n is the size of the matrix. Therefore, the whole process needs to compute $n \times n \times n$ multiplications. **In the basic algorithm, the n^3 multiplications are processed in series**, so when the size of the matrix n is very large, it takes a lot of time.

In order to realize the parallel version of matrix multiplication algorithm **based on OpenCL**, we set up a workspace with global size of n^3 , meaning there are n^3 **work-items to perform n^3 multiplications**. Therefore, we can improve the calculation speed through the parallel processing of the multiplications. At the same time, we divide different numbers of work-groups. **The number of work-items in each work-group is $n \times n$** . After the multiplication stage is completed (the next step is carried out after all the other multiplications are completed since **a barrier is used**), the reduction addition in each work-group is carried out by adding each multiplication result to the memory at the beginning of that same row. Finally, the results are transmitted back to the host memory for further calculation.

The result received by the host is a matrix of size $n \times n \times n$, where page i stores the reduction result after the dot multiplication of the matrix A 's i row and every single element in matrix B . The work-item position of every j row in this page i has a reduction result, which corresponds to the one before the block matrix is

added. The last step is to find the results of each row of work-item positions and add them to get c_{ij} .

The advantage of this method is that **the multiplication operation is completely parallel**, thus saving the overall operation time of the program. **However, an intermediate result matrix of $n \times n \times n$ size will be generated, which will occupy a large amount of memory space.** Actually, when the matrix size is too large (size 1024*1024), this first code's execution results in a segmentation fault.

For this reason, we changed the algorithm for a matrix of 1024 size: **the $n \times n \times n$ multiplication is divided into n cycles**, and the parallel operation of $n \times n$ multiplication is carried out each time, **so that the size of the middle matrix will be reduced to $n \times n$** , with the advantage of saving memory but **the drawback of taking more execution time than the original.**

Results

Table 1: Execution time for the different configurations

Matrix size	Threads count	Execution time (ms)
64	1	7,583
	2	6,776
	4	5,964
	8	6,275
	16	5,101
128	1	24,778
	2	20,486
	4	13,345
	8	13,753
	16	14,383
256	1	172,380
	2	132,940
	4	69,574
	8	70,416
	16	77,460
512	1	1365,270
	2	813,787
	4	544,878
	8	542,631
	16	459,460
1024	1	22154,758
	2	5873,911
	4	10130,578
	8	3306,972
	16	2719,110

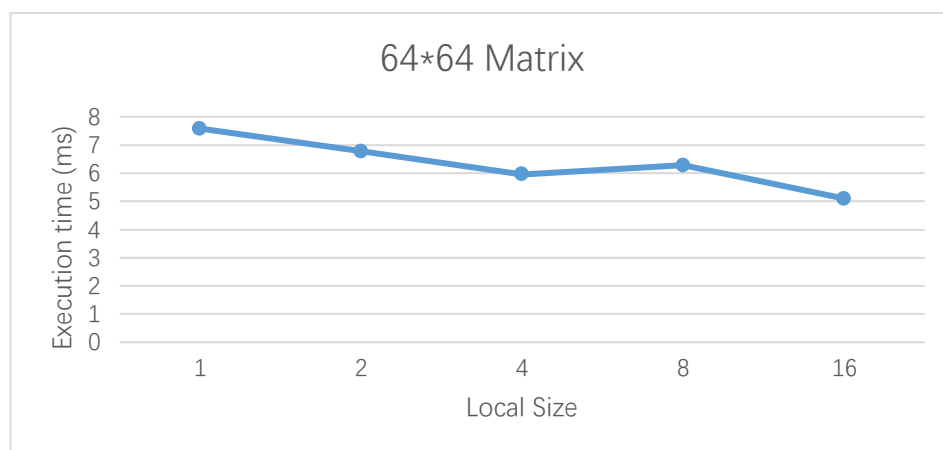


Figure 1: Execution time for a 64*64 matrix

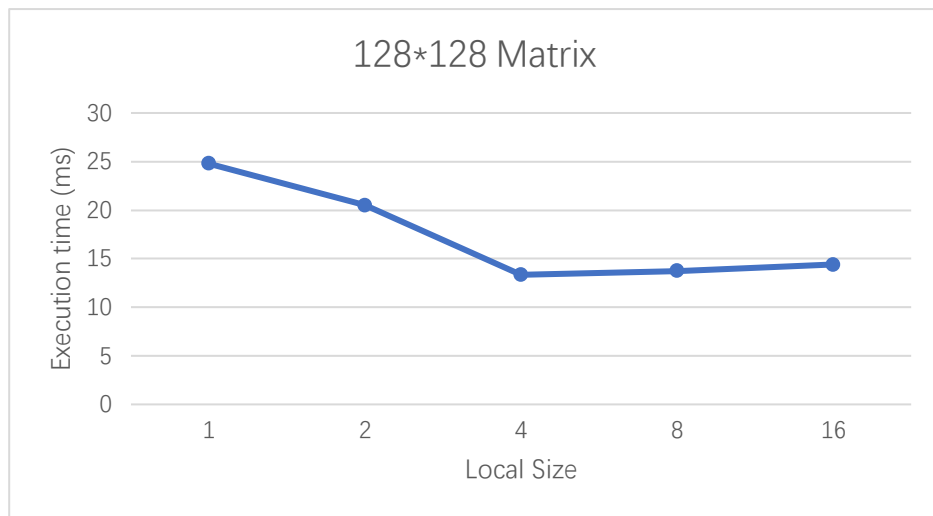


Figure 2: Execution time for a 128*128 matrix

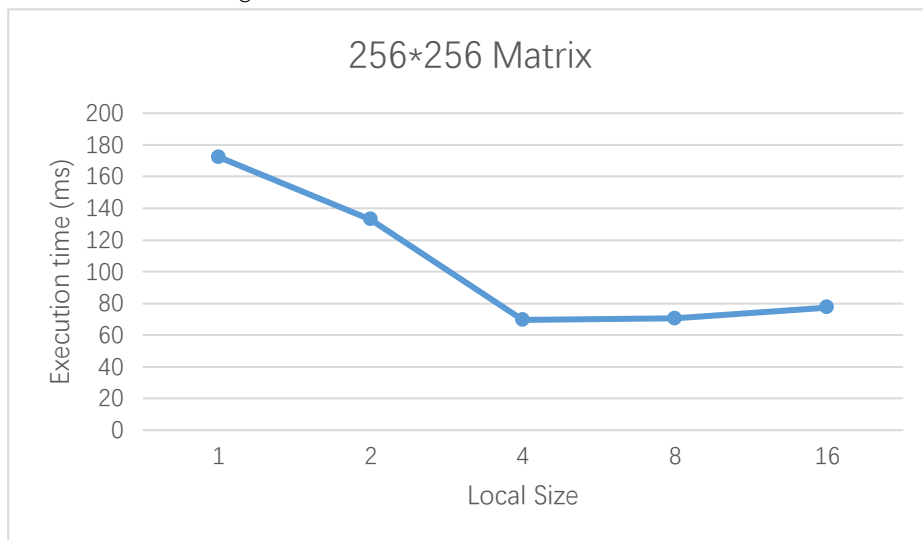


Figure 3: Execution time for a 256*256 matrix

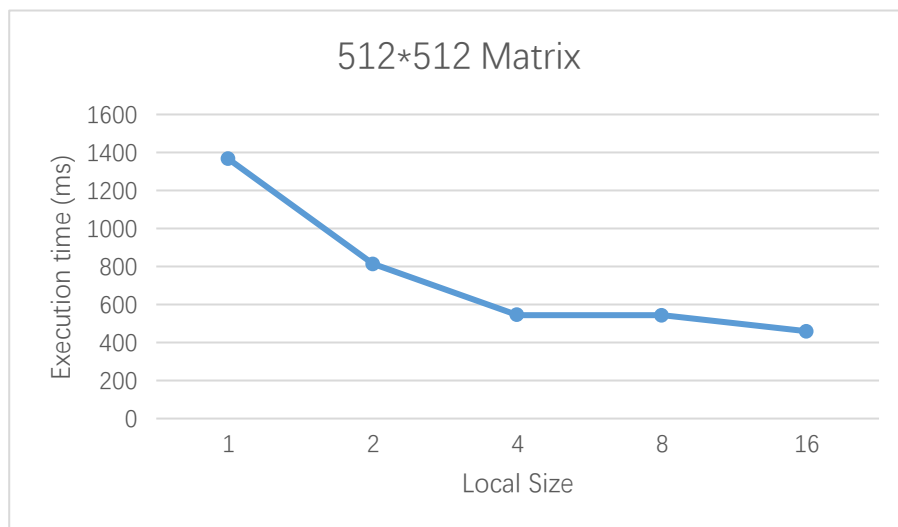


Figure 4: Execution time for a 512*512 matrix

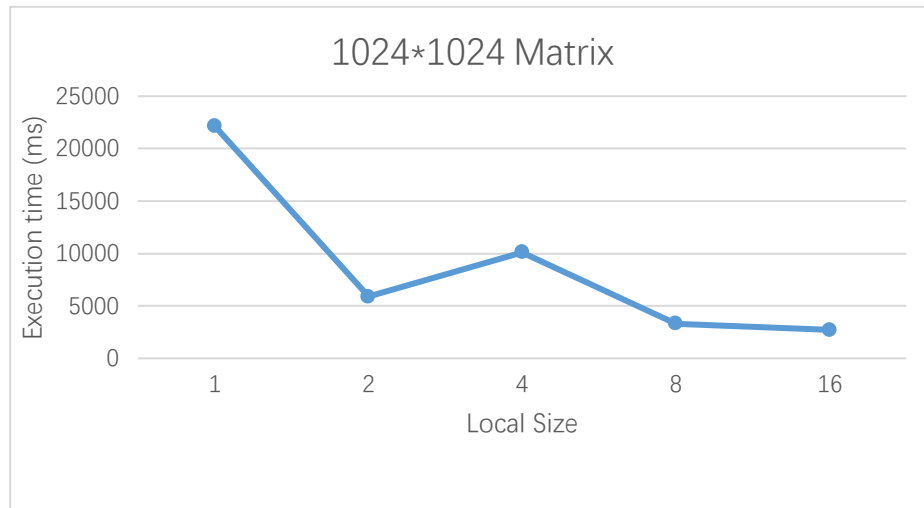


Figure 5: Execution time for a 1024*1024 matrix

For the presented results, the matrix multiplications with size $n = 64, 128, 256$ and 512 are calculated using the first implementation of the parallel algorithm, whereas for size $n = 1024$ it is calculated using the modified parallel algorithm.

As expected, the overall run time decreases as the local size increases, with the exception of a slight increase from 8 to 16 for matrix size 64. This may happen because, when a certain global size is fixed, **changing the amount of work-items per work-group can affect the performance and execution time of the program.** Generally speaking, a large local size will lead to higher parallelism, effectively exploiting parallel resources on hardware and improving the operation efficiency. However, **if the local size is too large for a relatively small global size, it may lead to resource competition and increased delay, thus slightly increasing the running time.** On the other hand, when there are enough computing tasks, such as for larger matrix sizes, these effects will be gradually drowned out.

Another algorithm and its Results

The modified algorithm only uses the global space of $n \times n$. For each work-item $[gidx, gidy]$, the dot product of the i row of matrix A and the j column of matrix B is calculated to get the elements of the resulting matrix C. Since this algorithm just uses a two-dimensional work space, it needs fewer work-items than the previous algorithm, which saves a load of invoking time. This is clearly translated into a much lower overall operation time.

The only odd point is that at a matrix size of 1024, for the local size of 4, the operation time is slightly higher than the normal trend. This is perhaps because of the 3-dimensional argument in the *clEnqueueNDRangeKernel* function (see Figure 6 below). The variable *global [2]* is set to 1, configuring it for a two-dimensional problem, but we ignore whether if this is may be the reason or not.

```
// Enqueue kernel for execution
global[0] = values;
global[1] = values;
//global[2] = values;
global[2] = 1;

//WORKITEMS = ELE_threads[j_all];
local[0] = (values > WORKITEMS) ? WORKITEMS : values;
local[1] = (values > WORKITEMS) ? WORKITEMS : values;
//local[1] = 1;
local[2] = 1;

ret = clEnqueueNDRangeKernel(queue, kernel, 3, NULL,
global, local, 0, NULL, NULL);
```

Figure 6: Possible explanation for the problem of the second algorithm

Table 2: Execution times for the second algorithm

Matrix size	Threads count	Execution time (ms)
64	1	7,308
	2	6,039
	4	2,791
	8	2,467
	16	2,387
128	1	26,902
	2	18,426
	4	12,031
	8	9,145
	16	9,108
256	1	182,574
	2	133,28
	4	77,759
	8	75,058
	16	73,284
512	1	1171,155
	2	441,363
	4	205,646
	8	180,021
	16	179,059
1024	1	11554,539
	2	2869,911
	4	7728,558
	8	1484,286
	16	1449,518

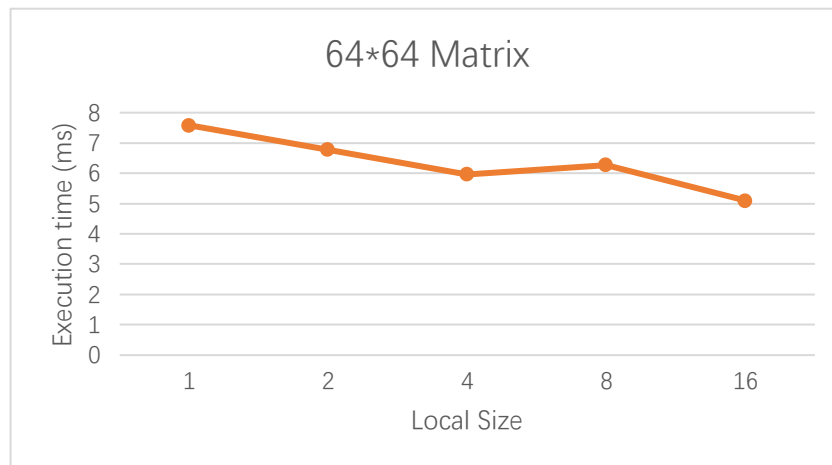


Figure 7: Execution time for a 64*64 matrix with the second algorithm

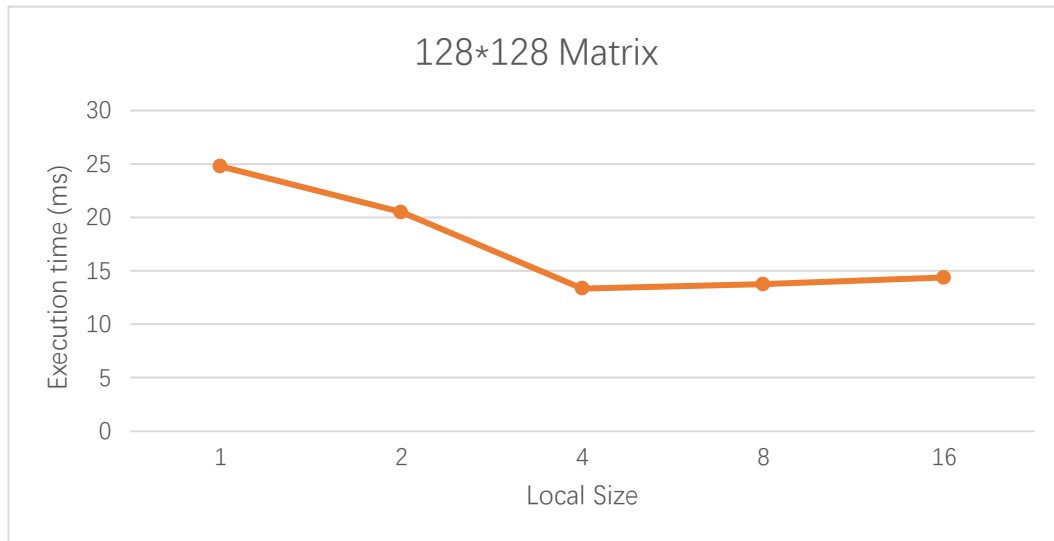


Figure 8: Execution time for a 128*128 matrix with the second algorithm

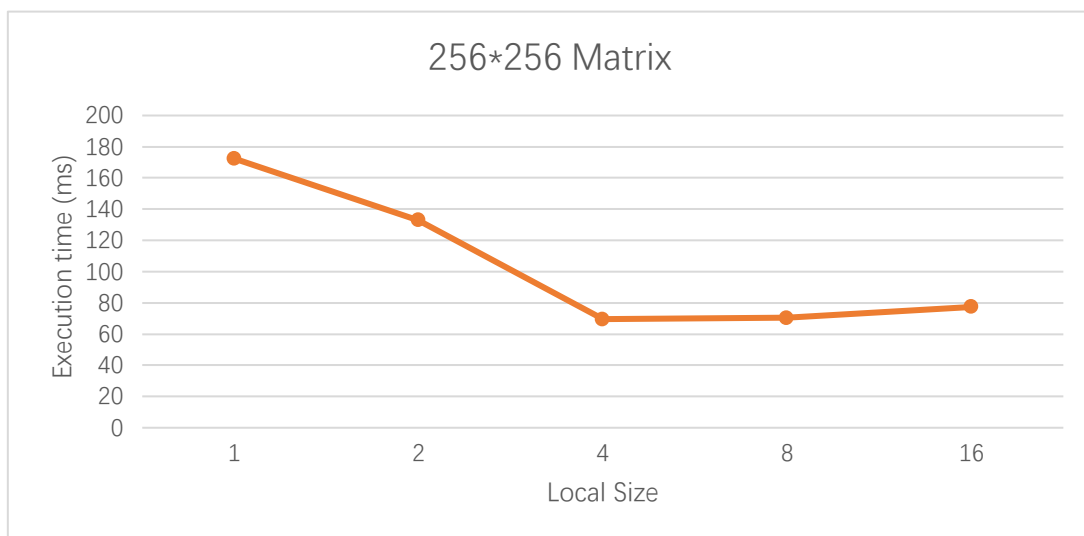


Figure 9: Execution time for a 256*256 matrix with the second algorithm

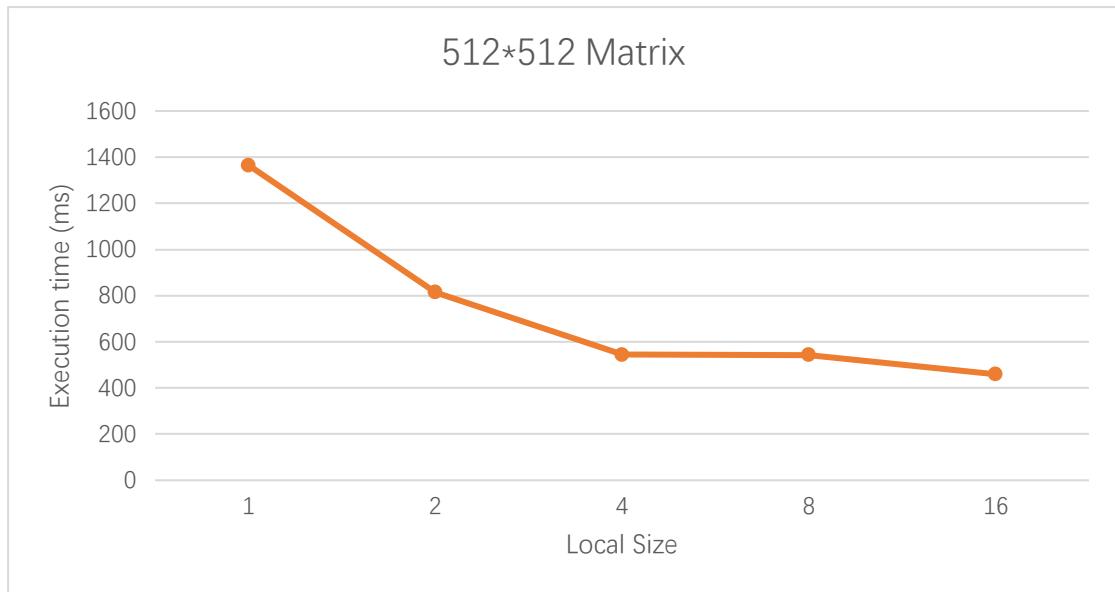


Figure 10: Execution time for a 512*512 matrix with the second algorithm

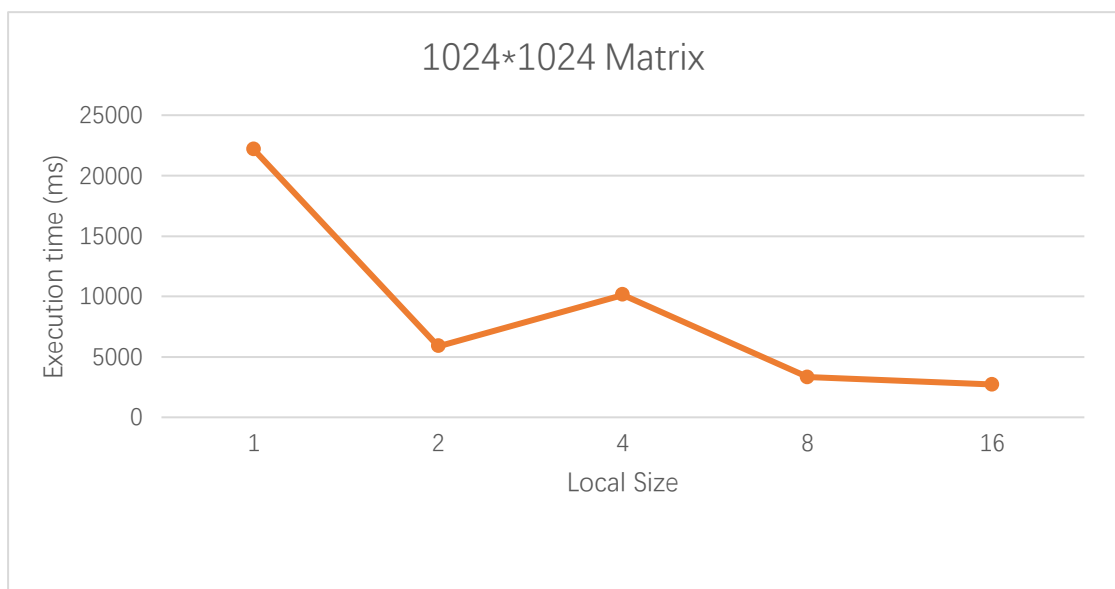


Figure 11: Execution time for a 1024*1024 matrix with the second algorithm